

## Guía Práctica de Laboratorio: Persistencia de Datos con JDBC

### 1. Contexto del Desafío

En el desarrollo de software empresarial, la capa de persistencia es el puente crítico entre nuestra lógica de negocio y la base de datos. Como estudiantes de séptimo semestre, el objetivo de esta práctica no es solo lograr que la información se visualice, sino **\*\*asegurar la estabilidad del sistema\*\***. Utilizaremos el estándar JDBC (Java Database Connectivity) para interactuar con PostgreSQL, enfocándonos en la gestión eficiente de recursos y el diagnóstico técnico de fallos.

### 2. Objetivos de Aprendizaje

Al finalizar este desafío, el estudiante será capaz de:

- Encapsular la lógica de conexión: Centralizar la configuración de la base de datos para facilitar el mantenimiento.
- Implementar `try-with-resources`: Entender cómo Java gestiona automáticamente el cierre de objetos `Connection`, `PreparedStatement` y `ResultSet`, evitando fugas de memoria.
- Diagnóstico Profesional: Interpretar los errores de base de datos no como un mensaje de texto plano, sino como un conjunto de metadatos (`SQLState`, `ErrorCode`) que permiten una resolución rápida de incidencias.

### 3. Fundamentos Teóricos (¿Qué deben saber?)

- **La gestión de recursos:**

En versiones antiguas de Java, era obligatorio cerrar cada conexión manualmente. Si ocurría un error y olvidábamos cerrar un objeto, el pool de conexiones de la base de datos se agotaba, causando que toda la aplicación fallara. La sentencia `try-with-resources` permite declarar los recursos que se deben cerrar al finalizar el bloque, garantizando que el sistema siempre quede limpio.

- El

## Diagnóstico de Errores

### (Log):

Cuando una consulta SQL falla, la mayoría de los desarrolladores junior solo ven `e.getMessage()`. Un desarrollador profesional analiza:

- **SQLState:** Un código estandarizado (5 caracteres) que nos dice qué tipo de error ocurrió (ej. error de sintaxis, falta de permisos, timeout de red).
- **ErrorCode:** Un código específico de la base de datos (PostgreSQL) que ayuda a encontrar la solución exacta en la documentación oficial.

#### 4. Desafío (Plantilla para el estudiante)

A continuación, complete los bloques indicados con ``TODO``.

```
`` `java
package Desafio_Individual;

import java.sql.*;

public class DesafioInventario {

    private static final String URL = "jdbc:postgresql://localhost:5432/TU_DB";
    private static final String USER = "TU_USUARIO";
    private static final String PASS = "TU_PASSWORD";

    // TODO: 1. Implementar método getConexion().
    // ¿Por qué? Para centralizar las credenciales y reutilizar la lógica.
    private static Connection getConexion() throws SQLException {
        // Implementar DriverManager.getConnection aquí
        return null;
    }

    public static void ejecutarConsulta() {
        String sql = "SELECT id_componente, stock_actual, umbral_minimo FROM
inventario";
```

```
// TODO: 2. Implementar la
estructura para manejar la conexión y la consulta.
// ¿Por qué? Evitar memory leaks usando try-with-resources.
try {

    // TODO: 3. Ejecutar la consulta y recorrer el ResultSet.
    // ¿Por qué? El ResultSet contiene los datos traídos desde el motor SQL.

} catch (SQLException e) {
    // TODO: 4. Implementar Log profesional.
    // ¿Por qué? Diagnosticar fallos mediante SQLState y ErrorCode.
}
}

public static void main(String[] args) {
    ejecutarConsulta();
}
}

...

```

## 5. Solución de Referencia (Para el Instructor)

```
```java
package Desafio_Individual;

import java.sql.*;

public class DesafioInventario {

    private static final String URL = "jdbc:postgresql://localhost:5432/TU_DB";
    private static final String USER = "TU_USUARIO";
    private static final String PASS = "TU_PASSWORD";

    private static Connection getConexion() throws SQLException {

```

```
        return  
        DriverManager.getConnection(URL, USER, PASS);  
    }  
  
    public static void ejecutarConsulta() {  
        String sql = "SELECT id_componente, stock_actual, umbral_minimo FROM  
inventario";  
  
        // Implementación con try-with-resources para cierre automático  
        try (Connection conn = getConexion();  
            PreparedStatement stmt = conn.prepareStatement(sql);  
            ResultSet rs = stmt.executeQuery()) {  
  
            while (rs.next()) {  
                System.out.println("ID: " + rs.getInt("id_componente") +  
                    " | Stock: " + rs.getInt("stock_actual") +  
                    " | Umbral: " + rs.getInt("umbral_minimo"));  
            }  
  
        } catch (SQLException e) {  
            // Diagnóstico técnico detallado  
            System.err.println("--- DIAGNÓSTICO DE ERROR ---");  
            System.err.println("Mensaje: " + e.getMessage());  
            System.err.println("SQLState (Tipo de error): " + e.getSQLState());  
            System.err.println("ErrorCode (Detalle del motor): " + e.getErrorCode());  
        }  
    }  
  
    public static void main(String[] args) {  
        ejecutarConsulta();  
    }  
}  
  
...
```

## **6. Reflexión Final**

Como futuros ingenieros, nuestro código debe ser predecible y resiliente. Este desafío les ha permitido ver que la programación de bases de datos no se trata solo de hacer SELECT, sino de gestionar la salud de la aplicación bajo cualquier circunstancia. Un software que no maneja bien sus errores es un software que no está listo para el mundo real.